Lenguaje SQL IV, vistas e indices

Alex Di Genova

09/06/2026

Contenidos

- Sentencias condicionales
- Consultas avanzadas
- Vistas
- Indices

Sentencias condicionales

SQL

SELECT

- **SELECT** pipeline

SELECT [DISTINCT] *select_header*

FROM *tablas*
WHERE *expresion_filtrado*
GROUP BY *expresion_de_agrupamiento*
HAVING *expresion_filtrado*
ORDER BY *expresion_orden*
LIMIT *contador*

OFFSET *indice*

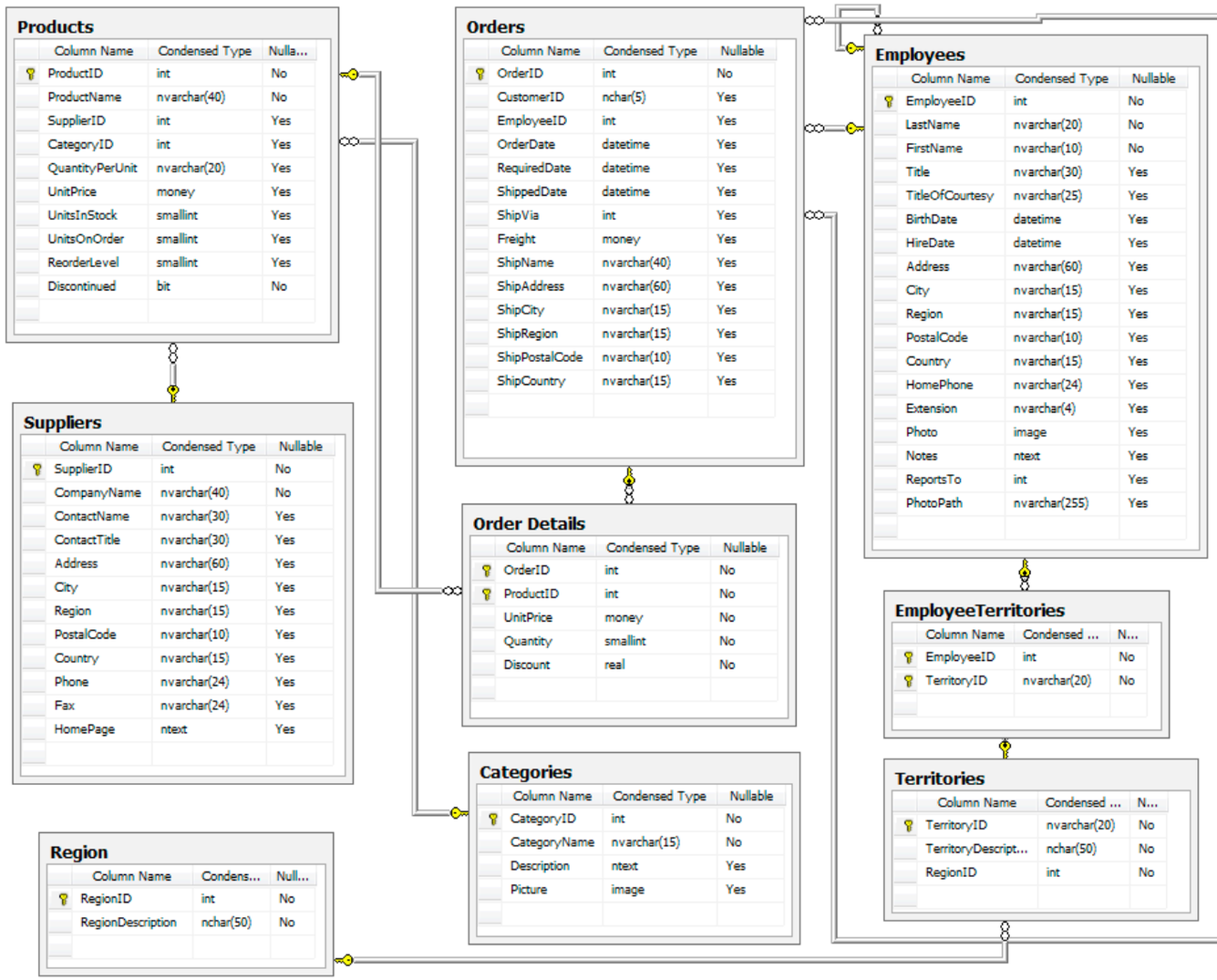
CASE

IF-THEN-ELSE

- Determine si los empleados son 'experimentados' o 'Junior' en función de la fecha de contratación

```
select LastName ||" "||FirstName "Nombre",HireDate as
"Fecha Contrato",
CASE WHEN HireDate < '2025-01-01' THEN 'Experimentado'
     WHEN HireDate >= '2025-01-01' THEN 'Junior'
     ELSE 'Otro' END AS "Estado"
FROM Employee
```

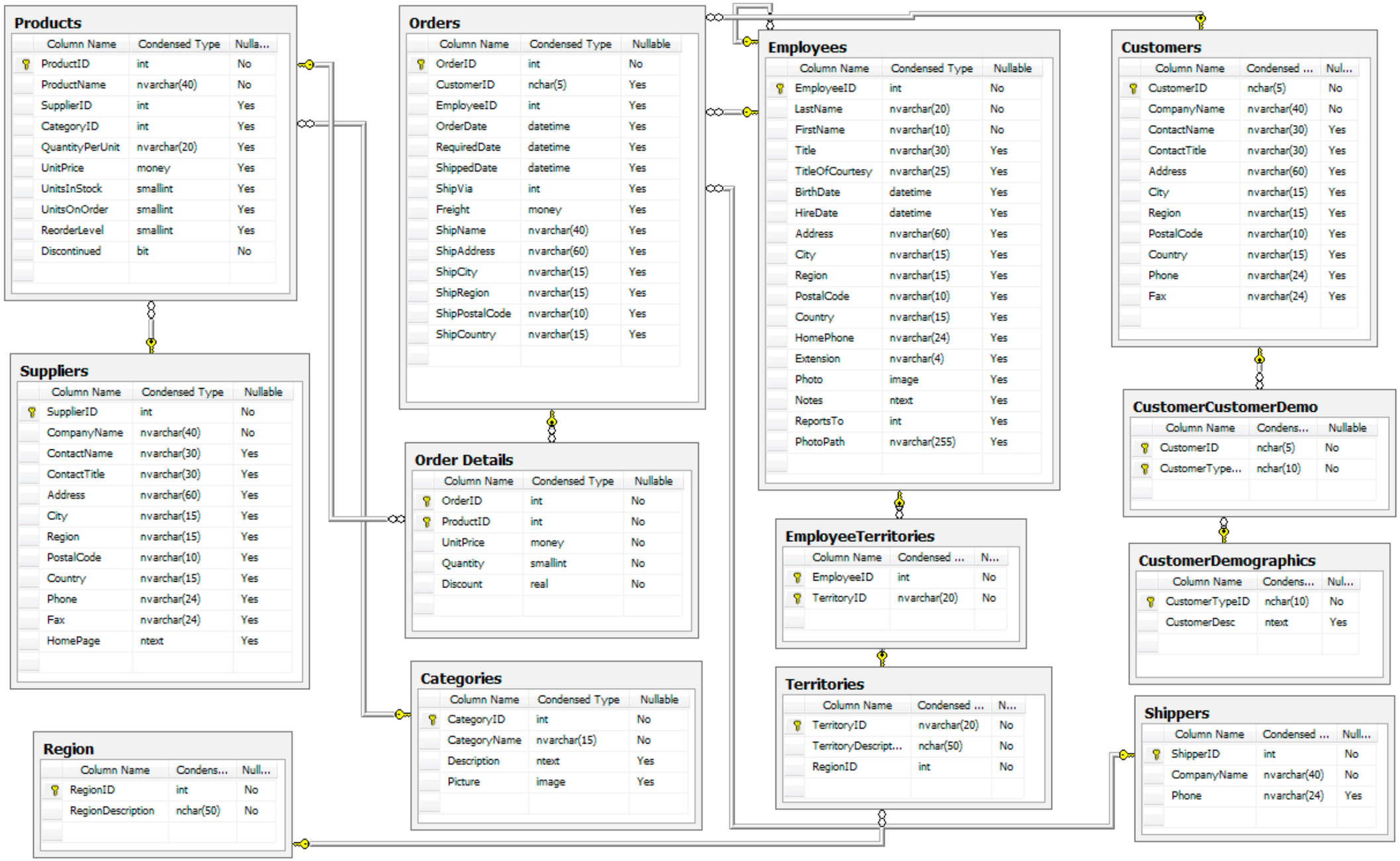
Nombre	Fecha Contrato	Estado
Davolio Nancy	2024-05-01	Experimentado
Fuller Andrew	2024-08-14	Experimentado
Leverling Janet	2024-04-01	Experimentado
Peacock Margaret	2025-05-03	Junior
Buchanan Steven	2025-10-17	Junior
Suyama Michael	2025-10-17	Junior
King Robert	2026-01-02	Junior
Callahan Laura	2026-03-05	Junior
Dodsworth Anne	2026-11-15	Junior



Consultas avanzadas

Consultas avanzadas

Bases de datos Empresa



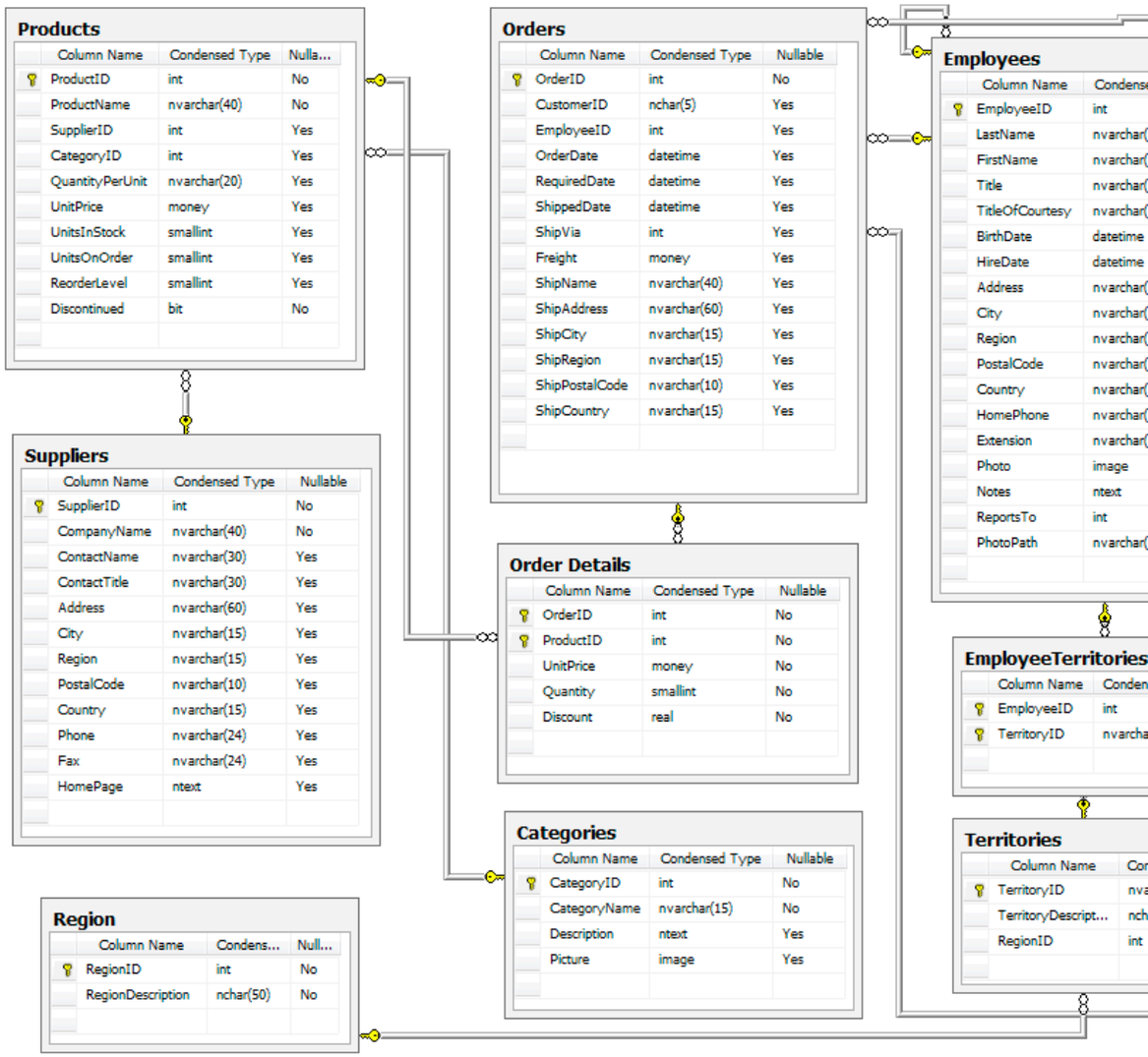
- La base de datos es para administrar clientes, pedidos, inventario, compras, proveedores, envíos y empleados de pequeñas empresas.

Ordenar resultados basado en una condición

- Ordenar los empleados por Ciudad si el cumpleaños es mayor a 1990 y por codigo postal en caso contrario.

```
SELECT FirstName "Nombre", LastName "Apellido",
BirthDate "Fecha Nacimiento", City "Ciudad",
PostalCode "Codigo Postal"
FROM Employee
ORDER BY
CASE WHEN Birthdate > '1990-01-01'
     THEN City
     ELSE PostalCode
    END
```

Nombre	Apellido	Fecha Nacimiento	Ciudad	Codigo Postal
Margaret	Peacock	1969-09-19	Redmond	98052
Nancy	Davolio	1980-12-08	Seattle	98122
Andrew	Fuller	1984-02-19	Tacoma	98401
Janet	Leverling	1995-08-30	Kirkland	98033
Michael	Suyama	1995-07-02	London	EC2 7JR
Robert	King	1992-05-29	London	RG1 9SP
Anne	Dodsworth	1998-01-27	London	WG2 7LT
Steven	Buchanan	1987-03-04	London	SW1 8JR
Laura	Callahan	1990-01-09	Seattle	98105



Cual es la catergoria de productos que vende más unidades?

- Cual es la categoria de productos que vende más unidades?

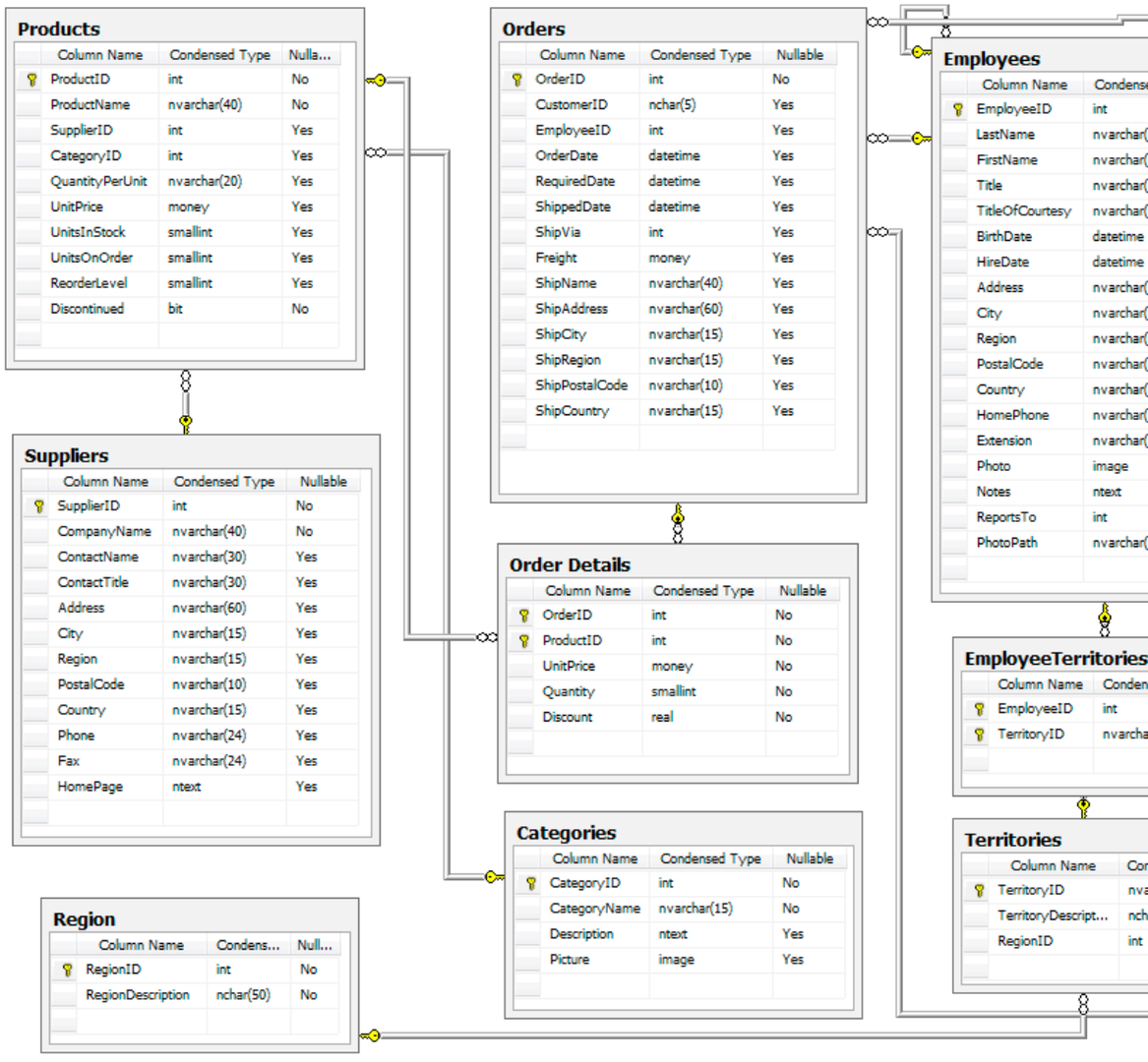
```
SELECT c.CategoryName "Categoria", SUM(o.Quantity) AS
"Total productos"
FROM OrderDetail o
JOIN Product p
ON o.ProductId = p.Id
JOIN Category c
ON p.Id = c.Id
GROUP BY 1
ORDER BY 2 Desc
```

Categoria	Total productos
Confections	209901
Condiments	209418
Grains/Cereals	206906
Meat/Poultry	205879
Produce	205437
Dairy Products	204578
Beverages	203353
Seafood	203223

- Cual es la categoria de productos que produce más dinero?

```
SELECT c.CategoryName "Categoria",
SUM(o.Quantity) "Total Productos",
SUM(o.Quantity * o.UnitPrice) "Total Vendido"
FROM OrderDetail o
JOIN Product p
ON o.ProductId = p.Id
JOIN Category c
ON p.Id = c.Id
GROUP BY 1
ORDER BY 3 Desc
```

Categoria	Total Productos	Total Vendido
Seafood	203223	8127800
Produce	205437	6162684
Meat/Poultry	205879	5146795
Dairy Products	204578	4500174.8
Grains/Cereals	206906	4416881.949999999
Condiments	209418	3977418.2
Beverages	203353	3659727.6
Confections	209901	2098810

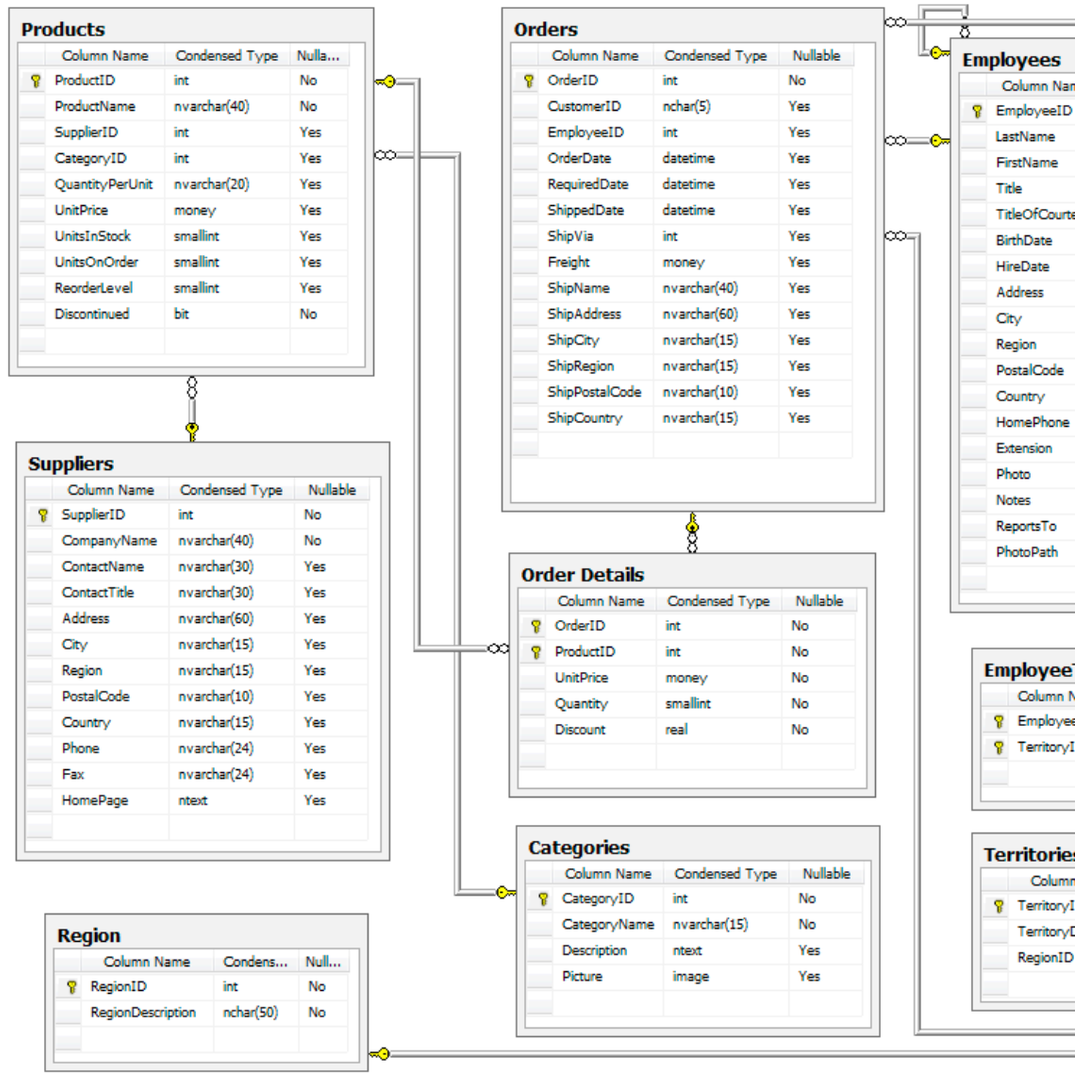


Determine los 10 productos más vendidos

- Determine los 10 productos más vendidos inluyendo precio promedio, descuento promedio y venta Total

```
SELECT p.Id "ID producto",
p.ProductName "Nombre",
c.CategoryName "Categoria",
    AVG(o.UnitPrice) AS  "Precio Promedio",
    AVG(o.Discount) AS  "Descuento Promedio",
    SUM(o.UnitPrice*o.Quantity*(1-o.Discount)) AS "Venta Total"
 FROM Product p
  JOIN OrderDetail o
   ON p.Id = o.ProductId
    JOIN Category c
    ON p.CategoryId = c.Id
GROUP BY p.Id
ORDER BY "Venta Total" DESC limit 10;
```

ID producto	Nombre	Categoria	Precio Promedio	Descuento Promedio	Venta Total
38	Côte de Blaye	Beverages	263.4479249011858	0.0001358695652173913	54009228.735
29	Thüringer Rostbratwurst	Meat/Poultry	123.7597387695445	0.000250244140625	25791923.041999985
9	Mishi Kobe Niku	Meat/Poultry	96.9975924547034	6.205013651030033e-05	19899792.5
20	Sir Rodney's Marmalade	Confections	80.98998763906057	9.147095179233625e-05	16816617.36
18	Carnarvon Tigers	Seafood	62.49071667285555	0.0002661220448075257	12940859.375
59	Raclette Courdavault	Dairy Products	54.97427058968361	0.00031392342730518274	11358970.7
51	Manjimup Dried Apples	Produce	52.988335982393934	0.00024452867098667317	11083309.65
62	Tarte au sucre	Confections	49.28288043477547	0.00032114624505928847	10185139.069999997
43	Ipoh Coffee	Beverages	45.9898180029513	0.00017215937038858828	9548562.7
28	Rössle Sauerkraut	Produce	45.58516681135679	0.0001550291454793501	9334526.240000056

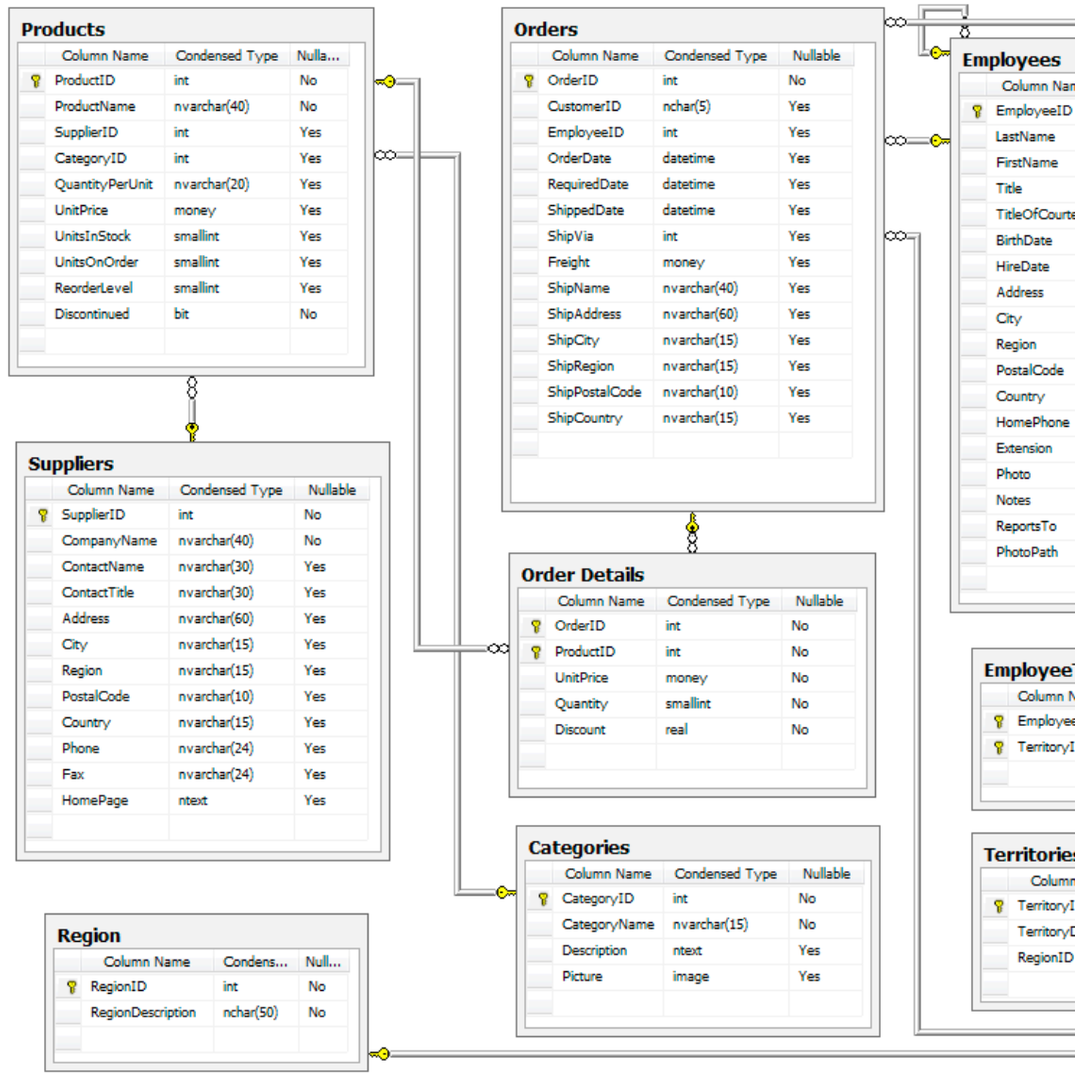


Cuales son los productos más caros por proveedor?

- Cuales son los productos más caros por proveedor?

```
SELECT s.companyname "Nombre Proveedor”,
 p.productname "Nombre Producto",
p.unitprice "Precio Unidad"
FROM supplier as s
JOIN product as p ON s.id = p.supplierid
WHERE p.unitprice = (SELECT MAX(p2.unitprice) FROM product
as p2 WHERE p2.supplierid = s.id)
ORDER BY p.unitprice DESC, s.companyname ASC, p.productname
ASC
 limit 10
```

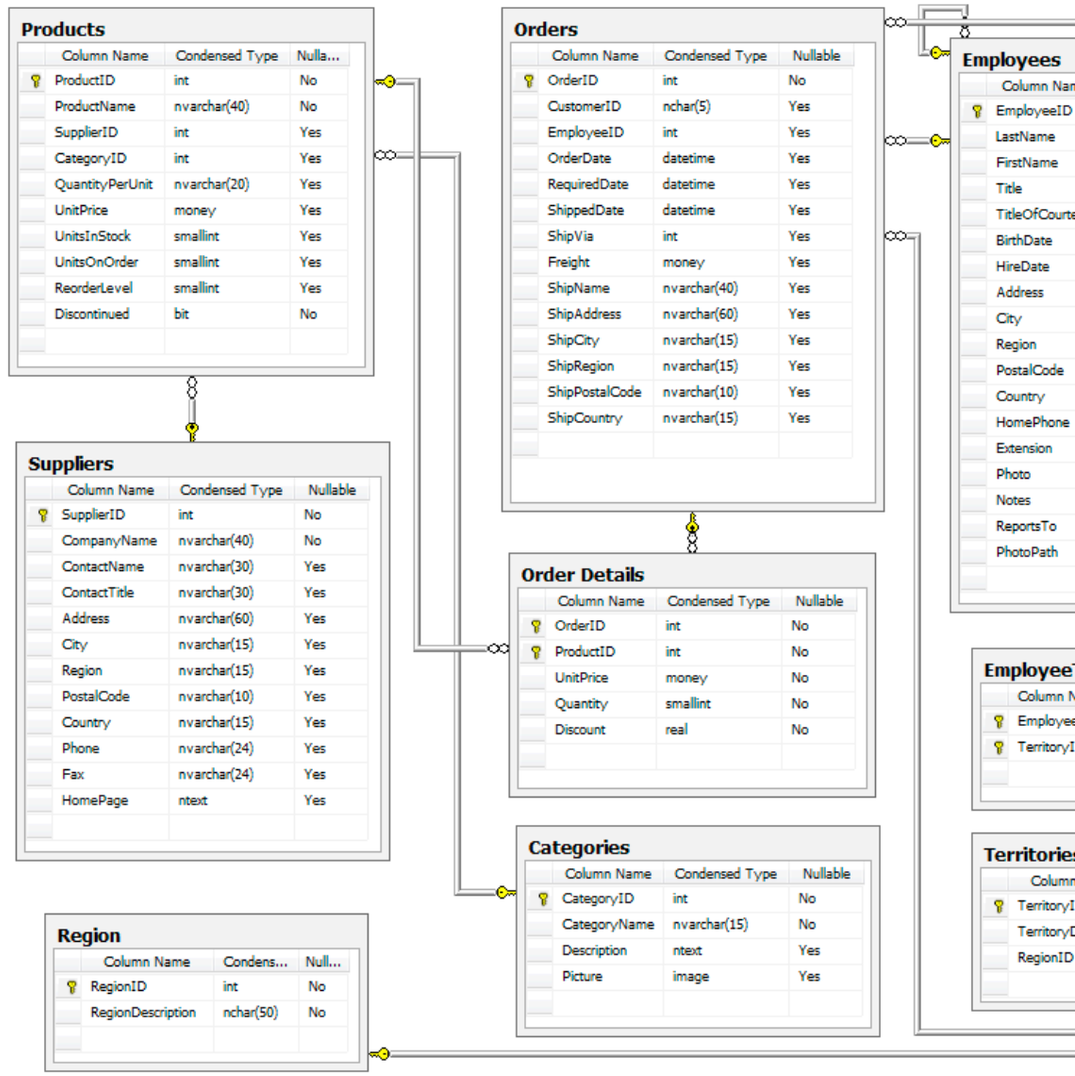
Nombre Proveedor	Nombre Producto	Precio Unidad
Aux joyeux ecclésiastiques	Côte de Blaye	263.5
Plutzer Lebensmittelgroßmärkte AG	Thüringer Rostbratwurst	123.79
Tokyo Traders	Mishi Kobe Niku	97
Specialty Biscuits, Ltd.	Sir Rodney's Marmalade	81
Pavlova, Ltd.	Carnarvon Tigers	62.5
Gai pâturage	Raclette Courdavault	55
G'day, Mate	Manjimup Dried Apples	53
Forêts d'érables	Tarte au sucre	49.3
Leka Trading	Ipoh Coffee	46
Heli Süßwaren GmbH & Co. KG	Schoggi Schokolade	43.9



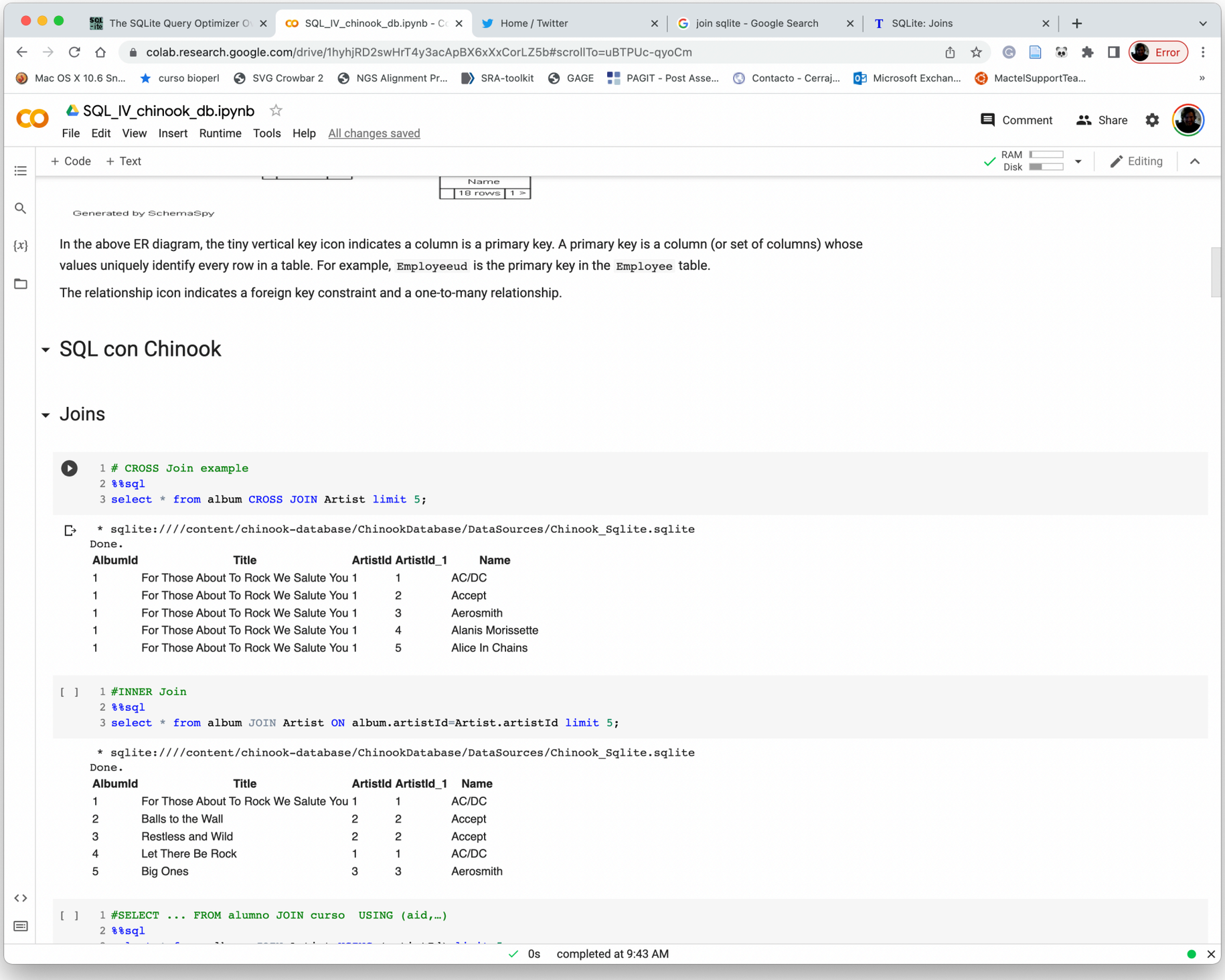
Que realiza la siguiente consulta?

- ?

```
select t.*,  t."Venta Total"-t."Venta Descuento Total" as
"Descuento Total" from (
select o.ProductID "ID producto",
sum(Quantity) "Unidades Total",
sum(Quantity*UnitPrice) "Venta Total",
sum(Quantity*UnitPrice*(1-Discount)) "Venta Descuento
Total" from OrderDetail as o
Group by  o.ProductId
ORDER by 4 DESC ) as t
limit 10;
```



Practicar en GoogleColab!!!



https://github.com/adigenova/uohdb/blob/main/code/SQL_VI_chinook_db.ipynb

Vistas

Vistas

Consultas con nombre

- Las vistas proporcionan una forma de empaquetar consultas en un objeto predefinido.
- Una vez creadas son similares a tablas de solo lectura.
    - CREATE VIEW nombre_vista AS SELECT consulta
- Las vistas son dinamicas, cada vez que son invocadas se ejecuta el SELECT otra vez
- Alternativamente son llamadas consultas con nombre.
- Usos:
    - Para almacenar consultas frecuentes o complejas.
    - Para crear versiones de tablas mas amigables al usuario (tiempo y fechas).
- DROP VIEW nombre_vista; # que pasa con los datos?

Vista de un Join anidado

Ejemplo

- Crear una vista de un join anidado

```
create view join_album_artista_track as select
* from artist natural join album join track
using (albumid)
```

- Como desplegamos los elementos de la lista?

```
select * from join_album_artista_track limit
10;
```

- Como eliminamos la vista?

```
drop view join_album_artista_track;
```

```
[7]  1 %%sql
     2 create view join_album_artista_track as select * from artist natural join album join track using (albumid)
```

```
 * sqlite:////content/chinook-database/ChinookDatabase/DataSources/Chinook_Sqlite.sqlite
Done.
[]
```

```
[8]  1 %%sql
     2 select * from join_album_artista_track limit 10;
```

```
 * sqlite:////content/chinook-database/ChinookDatabase/DataSources/Chinook_Sqlite.sqlite
Done.
```

ArtistId	Name	AlbumId	Title	TrackId	Name:1	MediaTypeId	GenreId	Composer
1	AC/DC	1	For Those About To Rock We Salute You	1	For Those About To Rock (We Salute You)	1	1	Angus Young, Malcolm Young,
2	Accept	2	Balls to the Wall	2	Balls to the Wall	2	1	None
2	Accept	3	Restless and Wild	3	Fast As a Shark	2	1	F. Baltes, S. Kaufman, U. Dirks Hoffman
2	Accept	3	Restless and Wild	4	Restless and Wild	2	1	F. Baltes, R.A. Smith-Diesel, S. Dirkscneider & W. Hoffman
2	Accept	3	Restless and Wild	5	Princess of the Dawn	2	1	Deaffy & R.A. Smith-Diesel
1	AC/DC	1	For Those About To Rock We Salute You	6	Put The Finger On You	1	1	Angus Young, Malcolm Young,
1	AC/DC	1	For Those About To Rock We Salute You	7	Let's Get It Up	1	1	Angus Young, Malcolm Young,
1	AC/DC	1	For Those About To Rock We Salute You	8	Inject The Venom	1	1	Angus Young, Malcolm Young,
1	AC/DC	1	For Those About To Rock We Salute You	9	Snowballed	1	1	Angus Young, Malcolm Young,
1	AC/DC	1	For Those About To Rock We Salute You	10	Evil Walks	1	1	Angus Young, Malcolm Young,

```
1 #drop view
2 %%sql
3 drop view join_album_artista_track;
```

```
 * sqlite:////content/chinook-database/ChinookDatabase/DataSources/Chinook_Sqlite.sqlite
Done.
[]
```

Indices

Indices

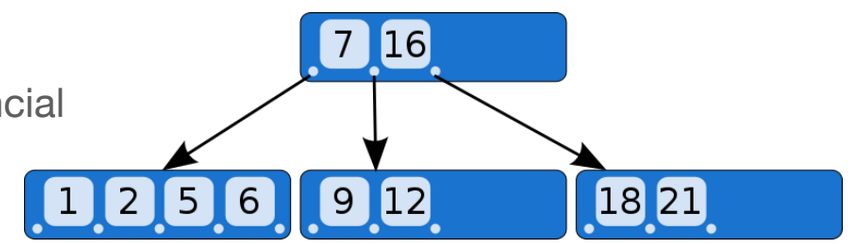
- Los indices sirven para optimizar consultas mediante el ordenamiento o indexación de uno o más columnas de una tabla.
    - La idea es encontrar filas sin la necesidad de revisar toda la tabla.
- Los indices requieren ser actualizados una vez que las tablas son modificadas.
    - CREATE [UNIQUE] INDEX nombre_index ON nombre_tabla (atributo1, atributo2…)
        - Los indices permiten valores duplicados pero la sentencia UNIQUE prohibe repeticiones.
- Existen algunos estandar para nombrar indices:
    - CREATE INDEX idx_alumno_nombre ON Alumno (nombre);
- DROP INDEX idx_alumno_nombre; # que pasa con la tabla alumno?
- Las claves primarias?

Indices

Optimización de consultas

- Sin indices, el motor de base de datos esta obligado de revisar las tablas completas en cada consulta.
    - Los JOINs son costosos coomputacionalmente.
    - La idea es encontrar filas sin la necesidad de revisar toda la tabla.
- Cada indice agrega una carga adicional a INSERT, UPDATE y DELETE.
    - Debemos evaluar donde colocar los indices.
- Los indices no se pueden crear en Vistas.
- Como almacena los datos el motor SQL?
- Por defecto, cada tabla es almacenada utilizando una estructura indexada (B-Tree SQLite).
    - A medida que se insertan filas en el B-Tree, las filas se ordenan, organizan y optimizan, de modo que una fila con un ROWID específico y conocido se puede recuperar de manera relativamente directa y rápida.

Permite: busquedas, inserciones, deleciones y accesso secuencial
Time(n)=O(log n)



- Cuando creamos un índice, el sistema de base de datos crea otro B-Tree para almacenar los datos del índice.
- El nuevo B-Tree se ordena y organiza usando la columna o columnas que se especifican en la definición del índice (! ROWID).

Indices

Ejemplo

- Creamos una tabla con 4 x 1000 numeros.

```
create table tbl (a,b,c,d);
INSERT INTO tbl (a, b,c,d)
VALUES
(84,39,78,79),
(182,39,67,153),
(83,166,143,188),
(145,205,380,366),
(317,358,70,303), …
```

```
1 %%time
2 %%sql
3 select * from tbl where a=29238;
```

```
 * sqlite:////content/chinook-database/ChinookDatabase/DataSources/Chinook_Sqlite.sqlite
Done.
CPU times: user 5.2 ms, sys: 0 ns, total: 5.2 ms
Wall time: 7.23 ms
```

a	b	c	d
29238	3171	23996	48794
29238	297097	765679	213680

```
[12]  1 %%time
      2 %%sql
      3 create index idx_tbl_a_b ON tbl(a,b)
```

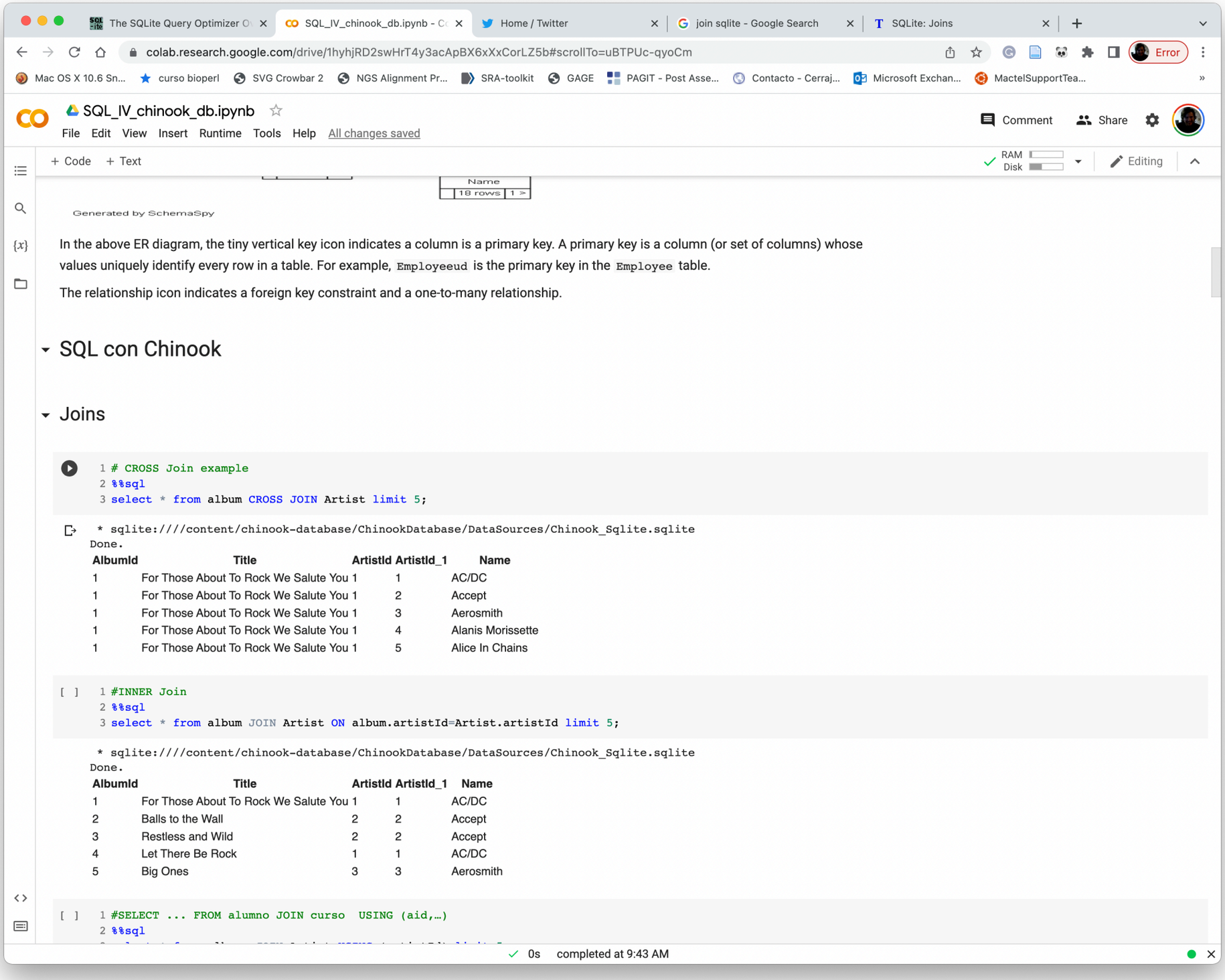
```
 * sqlite:////content/chinook-database/ChinookDatabase/DataSources/Chinook_Sqlite.sqlite
Done.
CPU times: user 9.09 ms, sys: 32 µs, total: 9.13 ms
Wall time: 22.4 ms
[]
```

```
1 %%time
2 %%sql
3 select * from tbl where a=29238;
```

```
 * sqlite:////content/chinook-database/ChinookDatabase/DataSources/Chinook_Sqlite.sqlite
Done.
CPU times: user 3.88 ms, sys: 0 ns, total: 3.88 ms
Wall time: 3.76 ms
```

a	b	c	d
29238	3171	23996	48794
29238	297097	765679	213680

Practicar en GoogleColab!!!



https://github.com/adigenova/uohdb/blob/main/code/View_index_chinook_db.ipynb

Consultas?

Consultas o comentarios?

Muchas gracias